

Comprehensive Deep Learning Notes: 3Blue1Brown Series

Based on Neural Networks Chapters 1-3

1 Video 1: Neural Networks Structure (Chapter 1)

1.1 Overview

Introduction to handwritten digit recognition using neural networks. This chapter covers the basic architecture: neurons, layers, weights, and biases, along with the mathematical foundation of forward propagation.

1.2 Key Concepts

1.2.1 Neurons and Activation

- **Neuron:** A computational unit that holds a number between 0 and 1
- **Activation:** The value inside a neuron, representing how “lit up” it is
- Input layer: 784 neurons (28×28 pixel grid)
- Output layer: 10 neurons (digits 0-9)
- Hidden layers: Arbitrary choice (2 layers with 16 neurons each in example)

1.2.2 Network Parameters

- Total neurons: $784 \rightarrow 16 \rightarrow 16 \rightarrow 10$
- Total weights and biases: $\approx 13,000$ parameters

1.3 Mathematical Foundation

1.3.1 Forward Propagation

For a neuron in layer $l + 1$:

Weighted Sum Computation:

$$z_j^{(l+1)} = \sum_{i=1}^{n_l} w_{ij}^{(l)} a_i^{(l)} + b_j^{(l)}$$

Where:

- $w_{ij}^{(l)}$: weight from neuron i in layer l to neuron j in layer $l + 1$
- $a_i^{(l)}$: activation of neuron i in layer l
- $b_j^{(l)}$: bias for neuron j in layer l
- $z_j^{(l+1)}$: weighted sum before activation function

Activation Function (Sigmoid):

$$a_j^{(l+1)} = \sigma(z_j^{(l+1)}) = \frac{1}{1 + e^{-z_j^{(l+1)}}}$$

Alternatively, ReLU (modern approach):

$$a_j^{(l+1)} = \max(0, z_j^{(l+1)})$$

1.3.2 Matrix Form

$$\mathbf{a}^{(l+1)} = \sigma \left(\mathbf{W}^{(l)} \mathbf{a}^{(l)} + \mathbf{b}^{(l)} \right)$$

Where:

- $\mathbf{W}^{(l)}$: weight matrix (rows = connections to one neuron in next layer)
- $\mathbf{a}^{(l)}$: activation vector from previous layer
- $\mathbf{b}^{(l)}$: bias vector
- σ : activation function applied element-wise

1.4 Why Layered Structure?

1. Layer 1 $\rightarrow$ 2: Detect edges and small patterns
2. Layer 2 $\rightarrow$ 3: Combine edges to recognize loops and lines
3. Layer 3 $\rightarrow$ Output: Recognize complete digits from patterns

1.5 Code Example

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def forward_pass(X, weights, biases):
    """Forward pass through network"""
    a = X
    for W, b in zip(weights, biases):
        z = np.dot(W, a) + b
        a = sigmoid(z)
    return a

# Network initialization
input_size = 784
hidden_sizes = [16, 16]
output_size = 10

weights = [np.random.randn(hidden_sizes[0], input_size) * 0.01]
for i in range(len(hidden_sizes)-1):
    weights.append(np.random.randn(
        hidden_sizes[i+1], hidden_sizes[i]) * 0.01)
weights.append(np.random.randn(output_size, hidden_sizes[-1]) * 0.01)

biases = [np.zeros((size,)) for size in hidden_sizes + [output_size]]
```

```
# Forward pass
output = forward_pass(image, weights, biases)
predicted_digit = np.argmax(output)
```

2 Video 2: Gradient Descent and Learning (Chapter 2)

2.1 Overview

This chapter introduces the cost function definition, gradient descent algorithm, and how networks learn through optimization.

2.2 Cost Function

2.2.1 Single Training Example

$$C_{\text{single}} = \sum_{k=1}^{10} (y_k - a_k^{(L)})^2$$

Where:

- y_k : desired output for digit k (1 if correct digit, 0 otherwise)
- $a_k^{(L)}$: actual output activation
- L : final layer
- Sum over all 10 output neurons

2.2.2 Total Cost Function

$$C_{\text{total}} = \frac{1}{n} \sum_{i=1}^n C_i$$

Where:

- n : number of training examples
- C_i : cost for i -th training example

2.3 Gradient Descent

2.3.1 Conceptual Framework

1. Start with random weights and biases
2. Compute gradient of cost function: ∇C
3. Update parameters in direction of negative gradient
4. Repeat until convergence

2.3.2 Gradient Vector

$$\nabla C = \left[\frac{\partial C}{\partial w_1}, \frac{\partial C}{\partial w_2}, \dots, \frac{\partial C}{\partial w_{13000}} \right]$$

Each component tells us:

- **Sign**: Whether to increase or decrease parameter
- **Magnitude**: How much that parameter affects cost (relative importance)

2.3.3 Update Rule

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial C}{\partial w}$$
$$b_{\text{new}} = b_{\text{old}} - \alpha \frac{\partial C}{\partial b}$$

Where:

- α : learning rate (small positive number, e.g., 0.01)
- $\frac{\partial C}{\partial w}$, $\frac{\partial C}{\partial b}$: partial derivatives (components of gradient)

2.4 Multivariable Calculus Connection

For 2D example:

$$\nabla C = \left[\frac{\partial C}{\partial x}, \frac{\partial C}{\partial y} \right]$$

- Points in direction of steepest ascent
- Negative gradient points downhill (minimization direction)
- Magnitude indicates steepness of slope

2.5 Code Example

```
def compute_cost(output, target):
    """Compute MSE cost"""
    return np.sum((target - output)**2)

def gradient_descent_step(weights, biases, X, y,
                           learning_rate):
    """Single gradient descent step"""
    # Forward pass
    activations = forward_pass(X[0], weights, biases)

    # Compute cost
    cost = compute_cost(activations, y[0])

    # Note: Gradient computation requires backpropagation
    # (see Chapter 3)

    return weights, biases, cost

# Training loop
for epoch in range(num_epochs):
    for batch_X, batch_y in mini_batches:
        weights, biases, cost = gradient_descent_step(
            weights, biases, batch_X, batch_y,
            learning_rate=0.01
        )
    print(f"Epoch {epoch}, Cost: {cost}")
```

2.6 Key Insights

- Network learns $\approx 96\%$ accuracy on test data with simple 2-hidden-layer architecture
- Can improve to 98% with tweaking
- **Important caveat:** Network doesn't necessarily learn patterns we expect
 - Second layer weights appear almost random
 - Network can confidently classify random noise
 - Cost function doesn't incentivize semantic features

2.7 Stochastic Gradient Descent (SGD)

Problems with full gradient descent:

- Computing gradient over ALL training examples is slow
- 10,000s of examples $\times$ 13,000 parameters = very expensive

Solution:

1. Randomly shuffle training data
2. Divide into mini-batches (e.g., 100 examples each)
3. Compute gradient using mini-batch only
4. Update weights
5. Repeat with next mini-batch

Effect:

- “Drunk man stumbling downhill” vs “carefully calculated descent”
- Quick steps with approximate gradients vs slow steps with exact gradient
- Usually converges faster in practice

3 Video 3: Backpropagation (Chapter 3)

3.1 Overview

This chapter covers how to efficiently compute gradients using the backpropagation algorithm, the chain rule application in neural networks, and visual intuition before mathematical formalism.

3.2 Key Concept: What Should Change?

3.2.1 Output Layer Perspective

For each output neuron, we want:

- Desired neuron: activation $\rightarrow 1$
- Other neurons: activation $\rightarrow 0$

Desired changes (nudges):

- If $\text{output}_k < \text{target}_k$: nudge UP
- If $\text{output}_k > \text{target}_k$: nudge DOWN
- Magnitude of nudge $\propto |\text{output}_k - \text{target}_k|$

3.2.2 Three Ways to Achieve Nudge

For neuron j with activation a_j :

$$z_j = \sum_i w_{ij} a_i + b_j$$
$$a_j = \sigma(z_j)$$

To increase a_j :

1. **Increase bias:** $\Delta b_j > 0$
2. **Increase weights:** $\Delta w_{ij} > 0$ (especially if a_i is large)
3. **Change previous activations:** $\Delta a_i > 0$ (for positive weights) or < 0 (for negative weights)

3.3 Backpropagation Algorithm (Intuitive)

3.3.1 Step 1: Output Layer

$$\text{error}_{\text{output}} = (a_{\text{output}} - y_{\text{target}})$$
$$\text{desired_change}_{\text{output}} = -2 \cdot \text{error}_{\text{output}}$$

3.3.2 Step 2: Propagate Backward

For each neuron in previous layer:

$$\text{desire}_i = \sum_j w_{ij} \cdot \text{desire}_j$$

Key insight: “Neurons that fire together wire together”

- Strong connections (large w_{ij}) propagate more desire
- Active neurons (large a_i) cause larger weight updates

3.3.3 Step 3: Update Weights and Biases

$$\Delta w_{ij} \propto \text{desire}_j \times a_i$$

Intuition:

- If neuron j needs to change a lot (large desire)
- AND neuron i is very active (large a_i)
- THEN strengthen connection w_{ij}

3.4 Backpropagation Algorithm (Mathematical)

3.4.1 Chain Rule Application

For cost C and weight w_{ij} :

$$\frac{\partial C}{\partial w_{ij}} = \frac{\partial C}{\partial a_j} \times \frac{\partial a_j}{\partial z_j} \times \frac{\partial z_j}{\partial w_{ij}}$$

Where:

- $\frac{\partial z_j}{\partial w_{ij}} = a_i$
- For sigmoid: $\frac{\partial a_j}{\partial z_j} = \sigma'(z_j) = \sigma(z_j)(1 - \sigma(z_j))$
- For ReLU: $\frac{\partial a_j}{\partial z_j} = 1$ if $z_j > 0$, else 0

Define: $\delta_j = \frac{\partial C}{\partial z_j}$ (error in z)

Then:

$$\begin{aligned} \frac{\partial C}{\partial w_{ij}} &= \delta_j \times a_i \\ \frac{\partial C}{\partial b_j} &= \delta_j \end{aligned}$$

3.4.2 Recursive Definition of δ

For output layer L :

$$\delta_j^{(L)} = (a_j^{(L)} - y_j) \times \sigma'(z_j^{(L)})$$

For hidden layer l :

$$\delta_j^{(l)} = \left(\sum_k w_{jk}^{(l)} \delta_k^{(l+1)} \right) \times \sigma'(z_j^{(l)})$$

Note: $\delta_k^{(l+1)}$ comes from next layer. This enables efficient backward pass.

3.4.3 Full Backpropagation Update

1. Forward pass: compute all $a^{(l)}$ and $z^{(l)}$
2. Compute output layer error:

$$\boldsymbol{\delta}^{(L)} = (\mathbf{a}^{(L)} - \mathbf{y}) \odot \sigma'(\mathbf{z}^{(L)})$$

where $\odot$ is element-wise multiplication

3. Backward pass (l from $L - 1$ down to 1):

$$\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l)T} \boldsymbol{\delta}^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$$

4. Compute gradients:

$$\frac{\partial C}{\partial \mathbf{W}^{(l)}} = \delta^{(l+1)} (\mathbf{a}^{(l)})^T$$
$$\frac{\partial C}{\partial \mathbf{b}^{(l)}} = \delta^{(l+1)}$$

5. Update parameters:

$$\mathbf{W}^{(l)} := \mathbf{W}^{(l)} - \alpha \frac{\partial C}{\partial \mathbf{W}^{(l)}}$$
$$\mathbf{b}^{(l)} := \mathbf{b}^{(l)} - \alpha \frac{\partial C}{\partial \mathbf{b}^{(l)}}$$

3.5 Code Example

```
def sigmoid_derivative(a):
    """Derivative of sigmoid: sigma'(z) = sigma(z)(1-sigma(z))"""
    return a * (1 - a)

def relu_derivative(z):
    """Derivative of ReLU"""
    return (z > 0).astype(float)

def backpropagation(X, y, weights, biases, activations,
                    z_values):
    """
    X: input (784,)
    y: target (10,)
    weights, biases: network parameters
    activations: list of a^(l) from forward pass
    z_values: list of z^(l) from forward pass
    """
    m = 1 # single example
    L = len(weights)

    # Initialize gradients
    dW = [np.zeros_like(W) for W in weights]
    db = [np.zeros_like(b) for b in biases]

    # Output layer error
    delta = ((activations[-1] - y) *
              sigmoid_derivative(activations[-1]))

    # Backpropagate
    for l in range(L-1, -1, -1):
        # Compute gradients for layer l
        dW[l] = np.outer(delta, activations[l])
        db[l] = delta

        # Propagate error to previous layer
        if l > 0:
            delta = (np.dot(weights[l].T, delta) *
                      sigmoid_derivative(activations[l]))

    return dW, db

def train_network(X, y, weights, biases, learning_rate=0.01,
                  epochs=1000):
```

```

"""
X: training data (n_samples, 784)
y: labels (n_samples, 10) - one-hot encoded
"""
for epoch in range(epochs):
    total_cost = 0

    for i in range(len(X)):
        # Forward pass
        activations = [X[i]]
        z_values = []
        a = X[i]

        for W, b in zip(weights, biases):
            z = np.dot(W, a) + b
            z_values.append(z)
            a = sigmoid(z)
            activations.append(a)

        # Compute cost
        cost = np.sum((a - y[i])**2)
        total_cost += cost

        # Backpropagation
        dW, db = backpropagation(X[i], y[i], weights,
                                   biases, activations, z_values)

        # Update weights and biases
        for l in range(len(weights)):
            weights[l] -= learning_rate * dW[l]
            biases[l] -= learning_rate * db[l]

    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Avg Cost: {total_cost/len(X)}")

return weights, biases

```

3.6 Mini-Batch and Stochastic Gradient Descent

3.6.1 Full Batch vs Mini-Batch

Full Batch:

$$\nabla C = \frac{1}{n} \sum_{i=1}^n \nabla C_i \quad [\text{average gradient over ALL samples}]$$

- Expensive: $O(13,000 \times n)$ per step
- Accurate direction but slow

Mini-Batch (typical size: 32-256):

$$\nabla C \approx \frac{1}{m} \sum_{i=1}^m \nabla C_i \quad [\text{average gradient over mini-batch}]$$

- Cheap: $O(13,000 \times m)$ per step (where $m \ll n$)
- Noisy gradient but fast convergence

- Good approximation in practice

Stochastic Gradient Descent (size 1):

$$\nabla C \approx \nabla C_i \quad [\text{gradient of single example}]$$

- Very fast: $O(13,000)$ per step
- Noisy but allows online learning

3.6.2 Algorithm

```
def train_sgd(X, y, weights, biases, learning_rate=0.01,
              batch_size=32, epochs=10):
    """Stochastic Gradient Descent with mini-batches"""
    n = len(X)

    for epoch in range(epochs):
        # Shuffle data
        indices = np.random.permutation(n)
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        # Process mini-batches
        for i in range(0, n, batch_size):
            X_batch = X_shuffled[i:i+batch_size]
            y_batch = y_shuffled[i:i+batch_size]

            # Accumulate gradients over mini-batch
            dW_batch = [np.zeros_like(W) for W in weights]
            db_batch = [np.zeros_like(b) for b in biases]

            for j in range(len(X_batch)):
                # Forward pass
                activations, z_values = forward_and_store(
                    X_batch[j], weights, biases
                )

                # Backpropagation
                dW, db = backpropagation(
                    X_batch[j], y_batch[j], weights,
                    biases, activations, z_values
                )

                # Accumulate
                for l in range(len(weights)):
                    dW_batch[l] += dW[l]
                    db_batch[l] += db[l]

            # Average and update
            for l in range(len(weights)):
                weights[l] -= (learning_rate/len(X_batch)) * dW_batch[l]
                biases[l] -= (learning_rate/len(X_batch)) * db_batch[l]

    return weights, biases
```

3.7 Why Backpropagation is Efficient

3.7.1 Naive Approach

For 13,000 weights, computing each $\frac{\partial C}{\partial w}$:

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial a_{\text{output}}} \times \frac{\partial a}{\partial z} \times \cdots \times \frac{\partial z}{\partial w}$$

Computing each derivative independently: $O(13,000)$

Computing all: $O(13,000^2) = O(169,000,000)$ per example!

3.7.2 Backpropagation Solution

Reuse intermediate computations:

1. δ from one layer reused to compute δ for previous layer
2. Share $\frac{\partial a}{\partial z}$ computations across multiple gradients

Complexity: $O(13,000)$ per example!

Speedup: $13,000\times$

4 Summary and Key Formulas

4.1 Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Range: $(0, 1)$

4.2 ReLU Function

$$f(z) = \max(0, z)$$
$$f'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

Range: $[0, \infty)$

Advantage: Easier to train, avoids vanishing gradient

4.3 Cost Function (MSE)

$$C = \frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - \mathbf{a}_i\|^2$$

4.4 Chain Rule (Backpropagation Core)

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w}$$

4.5 Full Forward Pass

$$\mathbf{a}^{(0)} = \mathbf{x} \quad (\text{input})$$
$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$
$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$
$$\mathbf{y}_{\text{pred}} = \mathbf{a}^{(L)} \quad (\text{output})$$

4.6 Full Backward Pass

$$\boldsymbol{\delta}^{(L)} = (\mathbf{y}_{\text{pred}} - \mathbf{y}_{\text{target}}) \odot \sigma'(\mathbf{z}^{(L)})$$
$$\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l+1)T} \boldsymbol{\delta}^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)}) \quad \text{for } l = L-1, \dots, 1$$
$$\frac{\partial C}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T$$
$$\frac{\partial C}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

4.7 Parameter Update

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha \frac{\partial C}{\partial \mathbf{W}^{(l)}}$$
$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial C}{\partial \mathbf{b}^{(l)}}$$

Where α is the learning rate.

5 Practical Considerations

1. **Learning Rate:** Too high $\Rightarrow$ divergence; too low $\Rightarrow$ slow convergence
2. **Weight Initialization:** Small random values (~ 0.01) avoid saturation
3. **Batch Size:** Trade-off between gradient accuracy and computational efficiency
4. **Epochs:** Typically 10-100 for simple networks; more for complex data
5. **Validation:** Always test on unseen data to check generalization
6. **Regularization:** Add penalty terms (L1/L2) to prevent overfitting
7. **Activation Functions:** Sigmoid (old), ReLU (modern), others for specific tasks

6 References

- Nielsen, M. A. (2015). Neural Networks and Deep Learning. *Online resource*.
- Olah, C. (n.d.). Chris Olah's Blog. Retrieved from <http://colah.github.io/>
- Distill. (n.d.). Distill. Retrieved from <https://distill.pub/>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.